

2D PLATFORMER

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

**Computer Science and Engineering
School of Engineering and Sciences**

Submitted by

Gnanasree | AP23110011650

Adi Lakshmi | AP23110011599

Himakshi | AP23110011596

Lakshmi Maheswari | AP23110011590



Under the Guidance of

Ms. V. Veda Sri

Lecturer, Department of CSE

SRM University-AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

[April, 2025]

Certificate

Date: _____

This is to certify that the work present in this Project entitled “**2D PLATFORMER**” has been carried out by **[Our team]** under my/our supervision. The work is genuine, original, and suitable for submission to the SRM University – AP for the award of Bachelor of Technology/Master of Technology in **School of Engineering and Sciences**.

Supervisor

Kavitha Rani

Designation,

Affiliation.

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, **Kavitha Rani**, Department of Computer Science & Engineering, SRM University, Andhra Pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work.

Gnanasree - AP23110011650

Adi Lakshmi - AP23110011599

Himakshi - AP23110011596

Lakshmi Maheswari - AP23110011590

Table of Contents

Certificate	i
Acknowledgements	iii
Table of Contents	v
Abstract	vii
1. Introduction	6
1.1 Heading 2	6
1.1.1 Heading 3	6
2. Methodology	7
2.1 Heading 2	7
2.1.1 Heading 3	8
3. Implementation	9
4. Result and Analysis	11
5. Discussion and Conclusion	12
6. Future Work	13
References	15

Abstract

The design and development of a 2D platformer game with Python and the Pygame module is presented in this project report. The main goal was to master fundamental programming principles including object-oriented design, event management, animation, and collision detection while producing an entertaining and playable game. Character movement, jumping, gravity simulation, monster encounters, collection objects, scoring, and level progression are some of the game's fundamental mechanics. Iterative enhancements based on user feedback and continuous testing were included into a streamlined Agile methodology.

The game's snappy user interface, realistic physics, and fluid controls are all made possible by modular programming and well-organized level design. Sprite animations and well-defined classes were used to construct player, platform, and opponent behaviors. Performance data showed a steady frame rate and effective memory use, and user testing validated the game's playability and engagement value. During development, issues like level difficulty balance, enemy AI, and collision handling were resolved.

In addition to meeting academic requirements, this project gave participants extensive practical game creation experience. It lays the groundwork for upcoming improvements like multiplayer support, cutting-edge visuals, and AI-driven gameplay while showcasing the application of computer science concepts in an entertaining and imaginative way.

1.Introduction

One of the most popular and pervasive kinds of digital entertainment nowadays is video games. Platformer games are unique among genres because of its straightforward physics, captivating gameplay, and nostalgic appeal. Guide a character through levels full of opponents, platforms, and obstacles while gathering stuff and aiming for a goal is a common feature in platformers. Both the player and the developer must be precise, time-sensitive, and creative when playing these games.

The development of a 2D platformer game with Python and the Pygame package is described in this paper. In order to understand and apply important programming ideas in an enjoyable and engaging manner, the project was conducted. Standard platformer elements like leaping, gravity, level progression, and collision detection are all present in the game.

In addition to creating a functional game, the project's objective was to gain an understanding of the entire game production process, from conception to design to implementation to final testing. It also functioned as a hands-on exercise in logic construction, object-oriented programming, and user experience design.

A variety of interdisciplinary abilities, including logic, mathematics, art design, and problem-solving, are required to develop a 2D platformer. Because of this, it offers a thorough educational experience as well as the opportunity to delve deeply into a variety of technical fields, such as event handling, animation, and frame rendering.

The design, methodology, implementation of various components, evaluation of results, difficulties encountered, conclusions reached, and recommendations for future improvements will all be covered in this report. The purpose of this documentation is to give a clear picture of the steps involved in creating a game from the ground up and how each stage affects the finished product.

2.Methodology

The approach used for this project is based on a streamlined form of Agile development, with a focus on incremental development, continuous testing, and user feedback. The primary phases were requirements collection, design and planning, prototype, implementation, testing, and documentation.

Gathering Necessities

Determining the features and mechanics of the game was the first step. Player controls (jumping, movement) were among the essential elements discovered.

- Physics and gravity simulation
- Platforms and barriers
- Advancement in level
- The identification of enemies and collisions
- Collectibles and scores
- Restart, game over, and start screen features

Design Phase

The locations of the player begins, passages, and adversaries were depicted in a simple graphic of the level layout. Each item was considered a separate class, including the player, opponent, and platform. A modular design was selected to simplify code management and future modifications.

Important classes are scheduled:

Player: Manages all states and actions pertaining to the player.

Surfaces that the player can stand on are represented by the platform

Level: Controls background, collectibles, and layout.

Tools and Technologies

- **Language:** Python (for its readability and large community support)
- **Library:** Pygame (for handling graphics, input, and sound)

- **IDE:** Visual Studio Code / PyCharm
- **Assets:** Free sprite sheets and tile maps from [OpenGameArt.org](https://opengameart.org/)

Development Cycle

Development followed a step-by-step approach:

1. **Basic movement and screen drawing**
2. **Gravity and collision detection**
3. **Adding platforms and environment**
4. **Enemy creation and behavior**
5. **Collectibles and scoring logic**
6. **UI components and screen transitions**

Regular testing made sure that errors were found early and promptly rectified. Peer review throughout testing also assisted in improving the gameplay.

3. Implementation

The most intricate and technical aspect of the project is the platformer game's implementation. In order to make the code readable and maintainable, the codebase was arranged according to object-oriented concepts.

Player Class

- This lesson covered: Left and right movement
- Jumping (which changes the y-axis velocity and is started by pressing a key)
- Gravity (a constant downward force that is modelled)
- Platform-based collision detection (to prevent falling)
- Using sprite sheets for sprite animation

Example code snippet:

python

```
if keys[pygame.K_SPACE] and self.on_ground:  
    self.velocity_y = -self.jump_strength
```

Platform and Environment

A list of Platform objects was used to represent platforms. Every platform has height, width, and coordinates. Bounding box detection was employed in player-platform collisions. Platforms were initialised in arrays or kept in level files.

Enemy AI

Basic opponents moved from left to right and had a patrol range. A collision with the player ended the game or resulted in health loss. The following was programmed into the enemy movement:

- Direction change dependent on a timer
- Detection of collisions with the player
- Cycling animation

Level Design

Tilemaps and arrays were used in the level design process. Platforms, obstacles, enemies, and collectibles were all represented by tiles. These were read by the game loop, which then represented them appropriately.

Tilemap sample:

csharp

CopyEdit

```
[0, 0, 1, 0, 0]
```

```
[0, 1, 0, 1, 0]
```

```
[1, 1, 1, 1, 1]
```

Game Loop

The game loop contained:

- Recording player input
- Changing the locations of objects
- Looking for collisions
- Sketching the player, platforms, adversaries, and surroundings
- displaying the game's status and score

User Interface:

The interface had a simple layout:

- ❖ Main menu with “Start” button
- ❖ Score display on top-left
- ❖ Game over screen with a “Restart” option

4. Result and Analysis

The finished game offered a playable, interactive experience and complied with the original design specifications. Among the important elements that were effectively included are fluid movement and responsive controls.

- Realistic physics behaviour with gravity
- Dynamic collision impacts and opponent movement
- Level scrolling and transitioning smoothly
- Grouping of items that influence the score
- Features for starting, pausing, and restarting games

User Testing

Several people on various platforms tested the game. Controls, difficulties, and graphic design feedback were gathered.

Performance Metrics

- On the majority of systems, FPS (frames per second) continuously remained above 50.
- Memory use stayed within reasonable bounds.
- After the final fixes, the game ran without crashes or significant issues.

Gameplay Evaluation

The game was tough and interesting to players. The feedback on collisions and jumping was generally accurate, and the controls were simple to use. Among the comments were:

Include additional visual effects.

- Add checkpoints to levels that are longer.
- For simpler stages, slightly slow down the enemy's speed.
- All things considered, the game was successful in providing a simple yet enjoyable experience. It works well as a starting point for later, more intricate platformers.

5. Discussion and Conclusion

Making this 2D platformer game gave me important knowledge about software engineering and game design. It was about creating an experience, not just writing code. Every choice, including movement speed and sprite selection, had an impact on the final gameplay.

Challenges Faced

- **Collision Detection:** The player would first become trapped on platforms. Axis-aligned bounding box (AABB) collision techniques were used to fix this.
- **Enemy AI:** Timer-based direction switching is necessary to create enemy patrol routes that are smooth and free of obstacles.
- **Level Complexity:** It was challenging to balance the difficulty. The game was frustrating since there were too many monsters and too simple stages.

Key Takeaways

- Modular coding is essential for controlling game complexity.
- How design can be significantly enhanced by user feedback.
- Frame timing is important for animation synchronization.
- Game development is a trial-and-error, iterative process.

Conclusion

The game's objective of becoming a playable, entertaining 2D platformer with necessary mechanics was accomplished. More significantly, it established the foundation for understanding essential ideas in visual design, physics simulation, and programming. This project has the potential to become a fully functional independent game with additional time and work.

6. Future Scope

The game's present version is fun and functional, but it might be improved in a few places to make it more sophisticated and appealing.

Graphics and Animation

- Increase the number of frame-based animations to make enemy and character movements more fluid.
- Use particle effects as you jump, gather coins, or pass away.

Sound and Music

- Level-related background music.
- Sound sounds for stage completion, coin gathering, enemy defeat, and jumping.

Level Design Tools

- Make a graphical user interface (GUI) for the level editor to facilitate level design and testing.
- Allow loading levels from CSV or JSON files that are external.

Enhanced AI

- Enemies that may follow, fly, or launch ranged attacks, among other behaviours.
- Include boss levels with unique features.

Power-Ups and Abilities

- Temporary invincibility, dash, double jump, or slowing down time.
- Include a health system that includes potions and hearts.

Multiplayer Mode

- Local multiplayer for two players that features cooperative or split-screen features.
- Sockets or frameworks such as PyGame Zero with networking are used for online multiplayer.

Save System

- Put checkpoints in place at long levels.
- Utilising external files or databases, save and load progress.

Platform Porting

- Export the game to a Windows exe file.
- Examine your possibilities for adapting the game to mobile platforms, such as Godot or Kivy.

The game can become more competitive and possibly publishable by putting these improvements into practice. This project might even be displayed at independent game expos or online gaming platforms with a more compelling plot, a well-designed user interface, and captivating gameplay loops.

References

1. Sweigart, A. (2015). *Making Games with Python & Pygame*. CreateSpace Independent Publishing
2. Pygame Developers. (n.d.). *Pygame Documentation*. Retrieved from <https://www.pygame.org/docs/>